
```

% Alex Masarie
% September 7, 2016
% Main function of point-to-point distance estimator.
% Important references are:
%     Dijkstra's algorithm -
%         Bertsimas, Dimitris and Tsitsiklis, John N. Introduction to
%         Linear Optimization. Athena Scientific, Dynamic Ideas:
%         Belmont, MA. 1997. 587 p.
%         Kirk, Joseph. DIJKSTRA Calculate Minimum Costs and Paths
%         using Dijkstra's Algorithm. See: dijkstra.m 2015
%     Google Earth -
%         Brown, Martin C. Hacking Google Maps and
%         Google Earth. Wiley Publishing Inc:Indianapolis,IN.
%         2006 373 p.
%         Google Earth software

% This function is designed to interface with a static Google maps
% dataset
% This is a network with fixed nodes and positive arc costs assigned
% by
% distance.

function [ dist,optimalPath ] = main_DistCalc( route,plotTOGGLE )
% Please consult README.txt for file type information.
% INPUTS:
%     route = numPts by 3 longitude latitude
%     plotTOGGLE = set to true(1) when we want to inspect plots
% OUTPUTS:
%     dist = cumulative distance of optimal route (curvature
%     corrected)
%     optimalPath = connection nodes of optimal route

disp('Welcome to main_DistCalc.m! Consult README.txt for further
info');
tic;

% Load roads MATLAB struct
load('C:\Users\Alex\Documents\MATLAB\ALOC8\Datasets\Omega\Roads\OUTPUT
\roads.mat');

% If not supplied, incorporate elevation
%TODO

% Initialize plots
numPts = size(route,1);
if plotTOGGLE
    gca; cla;
    xlabel('Longitude (deg NAD83)');
    ylabel('Latitude (deg NAD83)');
    title({'Optimal Routing';'with Dijkstra Algorithm'});
    grid on; hold on;
    set(gca, 'FontSize',13);

```

```

        xlim([-160.0 -60.0]);
        ylim([20 70]);
    end

    % Process route
    disp('    Adding route to network...');
    [ arcs1,nds1,E3 ] = processRoute(route,roads,plotTOGGLE);

    % Plot route points
    if plotTOGGLE
        for pp = 1:numPts
            scatter(route(pp,1),route(pp,2), 'bs');
        end
        getframe(); %Append figure
    end

    % Initiate Djisktra's algorithm between these points
    disp('    Computing optimal paths...');
    [costs,paths] = dijkstra( nds1,E3 );

    % By construction, the last several nodes have the info we need
    disp('    Post-processing results...');
    dist = zeros(numPts-1,1);
    optimalPath = cell(numPts-1,1);
    for pp = 1:numPts-1
        dist(pp,1) = costs(end-numPts+pp,end-numPts+pp+1); %Parse distance
        %traveled as cost
        optimalPath{pp,1} = paths{end-numPts+pp,end-numPts+pp+1}
        (:); %Parse optimal paths
        for kk = 1: numel(optimalPath{pp,1})-1
            aa = find( (E3(:,1) == optimalPath{pp,1}(kk,1)) &...
                (E3(:,2) == optimalPath{pp,1}(kk+1,1)) );
            if isempty(aa)
                aa = find( (E3(:,2) == optimalPath{pp,1}(kk,1)) &...
                    (E3(:,1) == optimalPath{pp,1}(kk+1,1)) );
            else
                aa = aa(1);
                if aa > numel(arcs1) || isempty(aa) %If this was a
                %negative trace
                    aa = find( (E3(:,2) == optimalPath{pp,1}(kk,1)) &...
                        (E3(:,1) == optimalPath{pp,1}(kk+1,1)) );
                    aa = aa(1);
                end
            end
            optimalPath{pp,1}(kk,2) = aa; %Connect arc info
            if plotTOGGLE
                plot(arcs1{aa}(:,1),arcs1{aa}(:,2), 'b-', 'LineWidth',1.15);
                scatter(nds1(optimalPath{pp,1}(kk,1),1),...
                    nds1(optimalPath{pp,1}(kk,1),2), 'rs');
                if kk == numel(optimalPath{pp,1}) - 1 %On last last
                    scatter(nds1(optimalPath{pp,1}(kk+1,1),1),...
                        nds1(optimalPath{pp,1}(kk+1,1),2), 'rs');
                end
                getframe();
            end
        end
    end

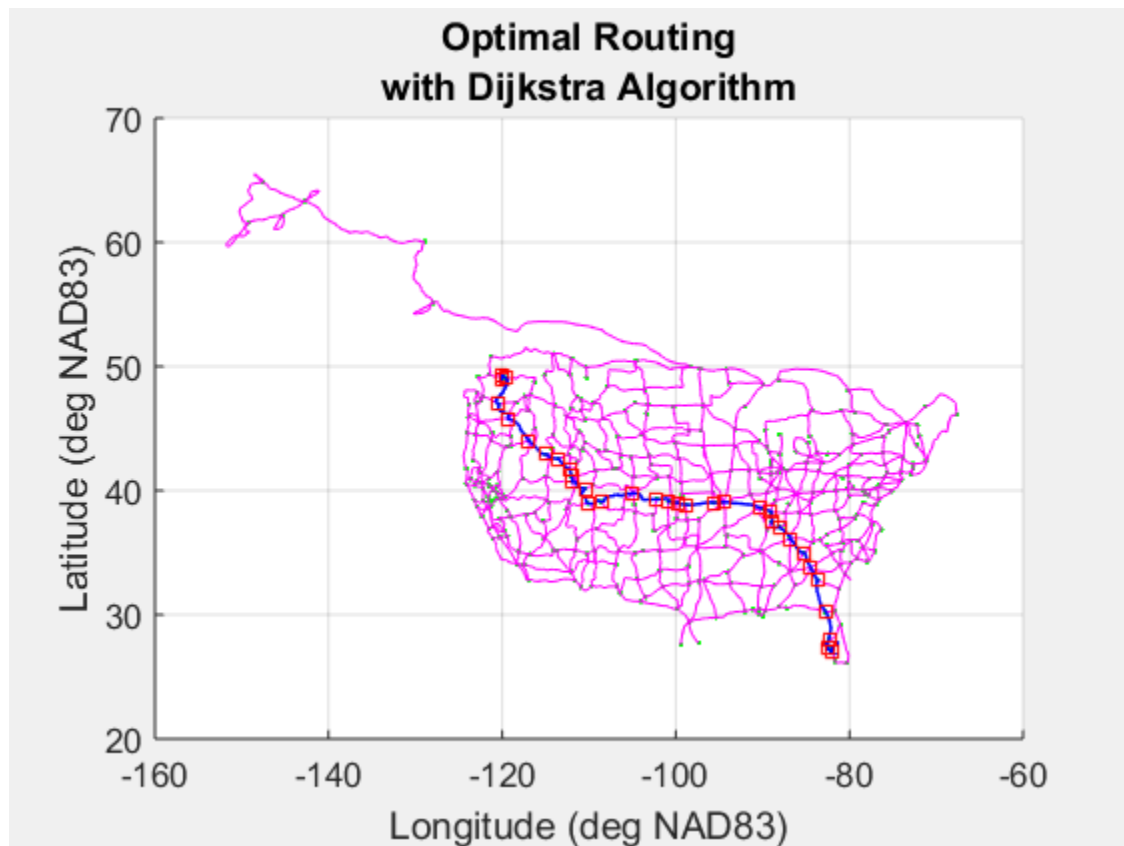
```

```

end
end
end
disp(['DONE, Runtime was ' num2str(toc) ' seconds']);
% Test against path integral model that travels gradients?
end

Welcome to main_DistCalc.m! Consult README.txt for further info
Adding route to network...
Computing optimal paths...
Post-processing results...
DONE, Runtime was 10.3907 seconds

```



Published with MATLAB® R2015a